

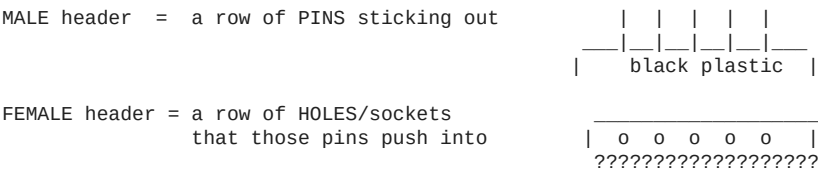
Braillix - Demo vs Final Product

How the wiring you build **now** differs from the wiring in the **finished product**, what each stage costs, and what to do today.
Written 2026-07-30. Companion to ELECTRONICS_BOM.md and ASSEMBLY_BIBLE.md.

PART 0 - First, what is a "header"?

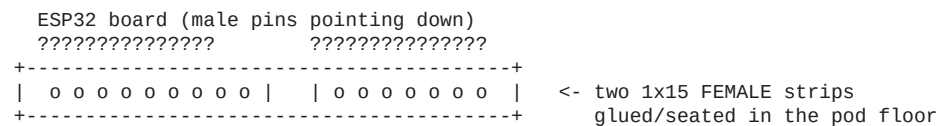
You said you don't know what this is, so here it is from scratch. It's simple.

A **header** is just a row of connector pins used to join two boards together. There are two halves, and they mate like a plug and a socket:



Your ESP32 already has male headers - those two rows of metal pins along its long edges, 15 per side. That's how it plugs into a breadboard.

A "1x15 female header strip" just means: one row, 15 sockets. Two of those, sitting in channels in the pod's floor, make a socket the ESP32 drops straight into:



Why bother? So the ESP32 is **removable**. You never solder the ESP32 down - you plug it in. If it dies, you pull it out and drop in a new one. Exactly like RAM in a computer: the slot is the female header, the stick has the male edge.

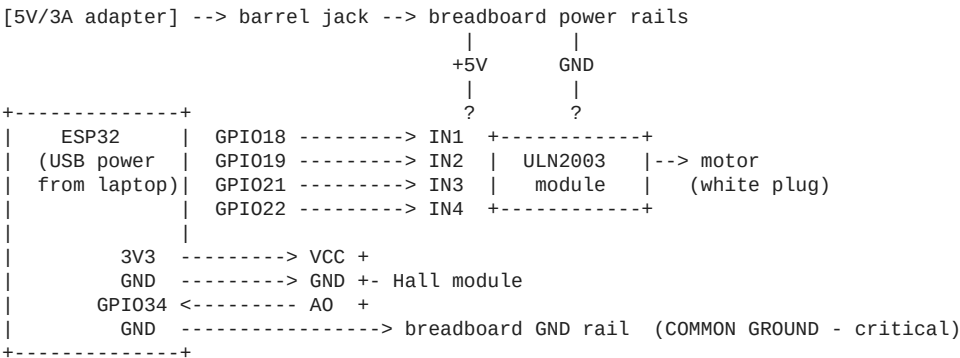
Check your ESP32 now: does it already have the pin rows soldered on? Most DevKits ship that way. If yes, you're set. If the pins came loose in a bag, you'd have to solder them on yourself (36 joints, 2.54mm spacing - doable at your skill level, but tedious).

You do NOT need female headers for the demo. The breadboard already *is* a big female header. You only need them when the ESP32 moves into the pod.

PART 1 - The three stages

[i] STAGE 1 - DEMO (now, on a breadboard)

Nothing printed, nothing soldered, nothing permanent.



- 13 wires, all Dupont jumpers. No solder.
- ESP32 powered by **USB from your laptop**; motor powered by the **adapter**.
- Their **grounds must be joined** - without that the hall sensor reads a constant 4095.
- Runs breadboard_test.ino -> live web dashboard on your phone.

What it proves: motor turns, hall finds home, dashboard works, text->braille->cam-number maths is right. That is a legitimate demo.

[OK] STAGE 2 - ONE ASSEMBLED CELL (when the resin arrives)

Exactly the same circuit. Same chips, same 13 connections, same firmware. The only change is that it stops living on a breadboard and gets stuffed inside the printed box.

```
+-----+ printed cell box (68x68x58mm) -----+
| top plate ? ? <- the six braille dots          |
| ? ?                                             |
|-----+-----|
| cam disc + 6 linkages + springs                 |
|-----+-----|
| 28BYJ-48 motor (bolted to base plate)          |
| hall sensor (next to the cam)                  |
|-----+-----|
| ULN2003 board, wires SOLDERED FLAT so it fits  |
|-----+-----+
| 4 motor wires + 3 hall wires                    |
| ? run out to the ESP32, still outside           |
```

What changes: the Dupont ends get cut off and soldered flat, because vertical Dupont headers make the ULN2003 ~20mm tall and the pocket is only 16mm. Soldered flat it is ~12mm.

What you buy for this stage: nothing electrical. Just solder, springs and screws.

? STAGE 3 - FINAL PRODUCT (multi-cell)

Here is the real architectural jump, and the reason the "custom board" exists.

The problem: one ESP32 driving one motor needs 4 wires. Five characters would need **20 wires** plus 5 hall sensors - and the ESP32 doesn't have enough pins, let alone a way to route 35 wires between snap-together bricks.

The solution: give every cell its own tiny brain, and let them all share **one 4-wire bus**.

```
[Brain Pod]      [Cell 1]      [Cell 2]      [Cell 3]
+-----+      +-----+      +-----+      +-----+
| ESP32 |<----->| muscle |<----->| muscle |<----->| muscle |
| WiFi  | pogo  | board  | pogo  | board  | pogo  | board  |
| buttons | 4 pin | + motor |      | + motor |      | + motor |
+-----+      +-----+      +-----+      +-----+
?              ?              ?              ?
+-- magnets hold the bricks together in a row -----+
```

The 4 pogo wires carry: +5V - GND - SDA - SCL
SDA+SCL are the I2C bus - one pair of wires talks to ALL cells.

Each muscle board has an **ATmega328P** that listens on I2C at its own address (0x20, 0x21...), receives "go to cam position 24", and drives its own ULN2003 locally. The ESP32 becomes a messenger, not a motor driver.

What you'd need for Stage 3 (NOT now):

Item	Why
Muscle board x N	One per cell. Order assembled (PCBA) - TQFP-32 at 0.8mm pitch is not hand-solderable.
Pogo connectors x (N+1)	4-pin, 2.54mm, spring-loaded, ~0.5A
2 x 4.7k Ω resistors	I2C pull-ups, at the ESP32 end only - the only discrete passives in the whole project
2 x 1x15 female headers	So the ESP32 plugs into the pod instead of being soldered
Nav caps + switches x 3	Pod front buttons

PART 2 - What the finished product looks like

A row of snap-together bricks, magnets holding them, pogo pins carrying power and data:

```
+-----+-----+-----+-----+-----+
| BRAIN | CELL 1 | CELL 2 | CELL 3 | CELL 4 |
| POD   |      |      |      |      |
| <| ( ) |> | ? ? | ? ? | ? ? | ? ? | <- braille dots on top
|      |      |      |      |      |
|      | ? ? | ? ? | ? ? | ? ? |
+-----+-----+-----+-----+-----+
64mm    68mm    68mm    68mm    68mm    = 336mm total
```

- **Brain Pod** - ESP32 inside, 3 nav buttons on the front, barrel jack on the lid, USB on the end
- **Each Cell** - motor + cam + 6 linkages + 6 dots, shows ONE character

- **Add or remove cells freely.** The pod counts them automatically by scanning the I2C bus.
- **Honest limitation:** characters sit 68mm apart because a 28mm motor per character cannot

physically fit at braille's 6.1mm reading pitch. This is a **braille teaching device**, not a compact reader. Own that in the presentation - it's a real design decision, not a flaw.

PART 3 - What actually changes between stages

	Stage 1 (demo)	Stage 2 (one cell)	Stage 3 (product)
Who drives the motor	ESP32 directly	ESP32 directly	Each cell's own ATmega
Wires ESP32->motor	4 Dupont	4 soldered	0 - goes over I2C
Wires between bricks	n/a	n/a	4 (pogo: 5V/GND/SDA/SCL)
Cells supported	1	1	any number
Connections	breadboard	soldered flat	pogo + magnets
ESP32 mounting	breadboard	loose	female headers in the pod
Resistors needed	0	0	2 x 4.7k?
Custom PCB	no	no	yes (order assembled)
Firmware	breadboard_test.ino	same	pod master + cell slave (not written)

The single most important line in that table: Stage 1 and Stage 2 use the **same circuit**. Building the demo is not throwaway work - it *is* the first cell.

PART 4 - What to do right now

This week (~\$1,100):

- 1 Buy: **60W temperature-controlled soldering station**, breadboard, USB-C **data** cable, 0.8mm 60/40 solder, wire strippers. *(Multimeter already owned.)*
- 2 [OK] **Jack polarity already measured and confirmed correct (2026-08-01), red = positive.** Note there is still no reverse-polarity protection in this circuit - wrong polarity kills the ESP32 instantly.

- 1 Build the Stage-1 breadboard circuit (13 wires, ASSEMBLY_BIBLE.md)
- 2 Flash breadboard_test.ino, confirm the dashboard on your phone
- 3 Rehearse the demo

When the resin arrives (~5 days):

- 1 Buy 2mm springs, 3x2mm homing magnet, screws, epoxy, calipers (~\$1,430)
- 2 **Measure the motor with the calipers** - 8 dimensions. This unblocks the cam and base plate.
- 3 Solder the ULN2003 wires flat; assemble Stage 2

After the presentation:

- 1 Fix the 4 remaining CAD defects (CAD_FIT_CHECK.md)
- 2 Only then think about muscle boards and pogo pins

Do not buy anything for Stage 3 yet. Pogo connectors, muscle boards and I2C resistors are all pointless until one cell physically works. Nothing in this project has ever moved a dot.